

Laboratorio #1 – Introducción a Wireshark

Reporte individual

Redes | Universidad del Valle de Guatemala | Semestre II - 2026

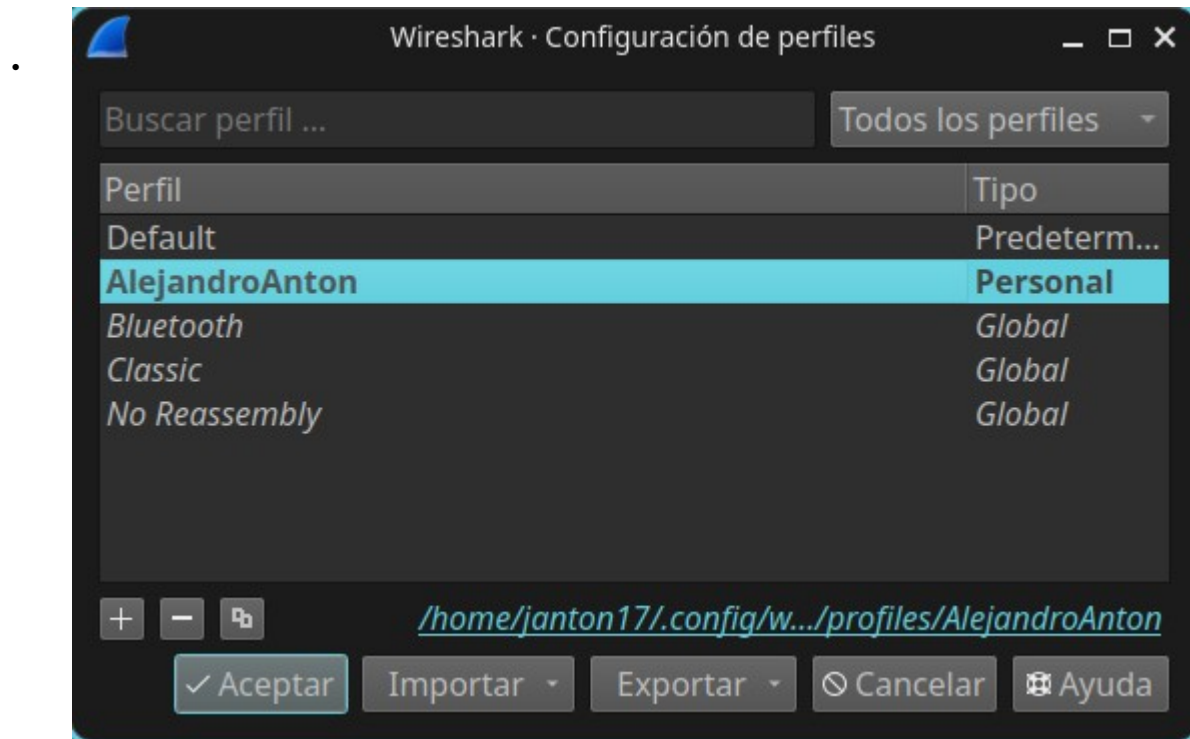
Nombre: Jose Alejandro Anton Escobar

Carnet: 221041

2. Personalización del entorno (3.4) y configuración de captura (3.5)

Estas dos secciones requieren evidencia (capturas de pantalla) tomadas directamente sobre tu instalación de Wireshark, por lo que debes completarlas de forma personal antes de entregar. A continuación se deja la guía de los pasos y comandos exactos para no perder tiempo buscándolos:

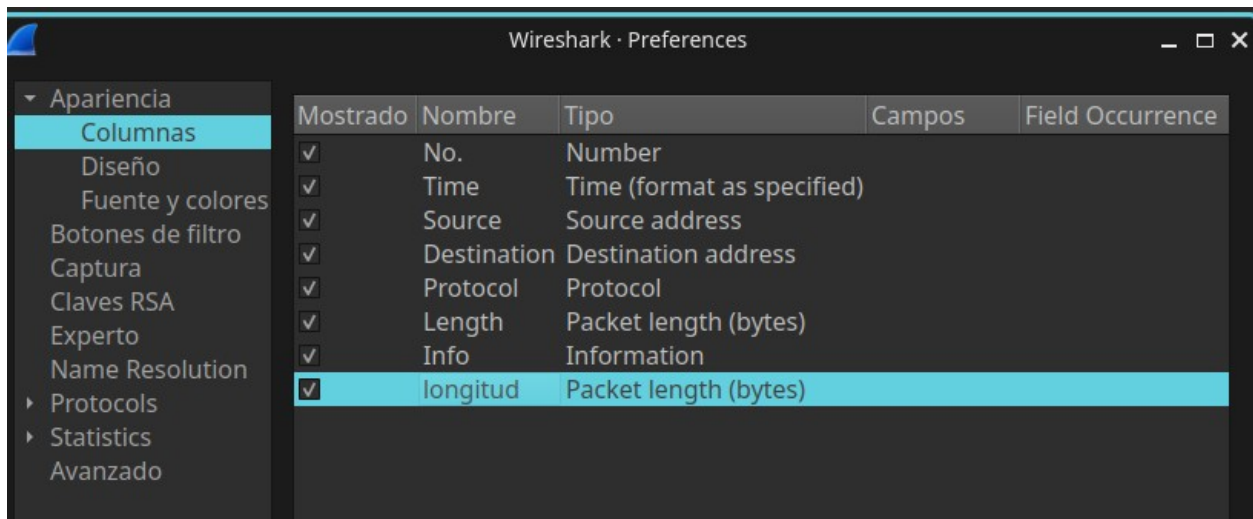
- Crear perfil:



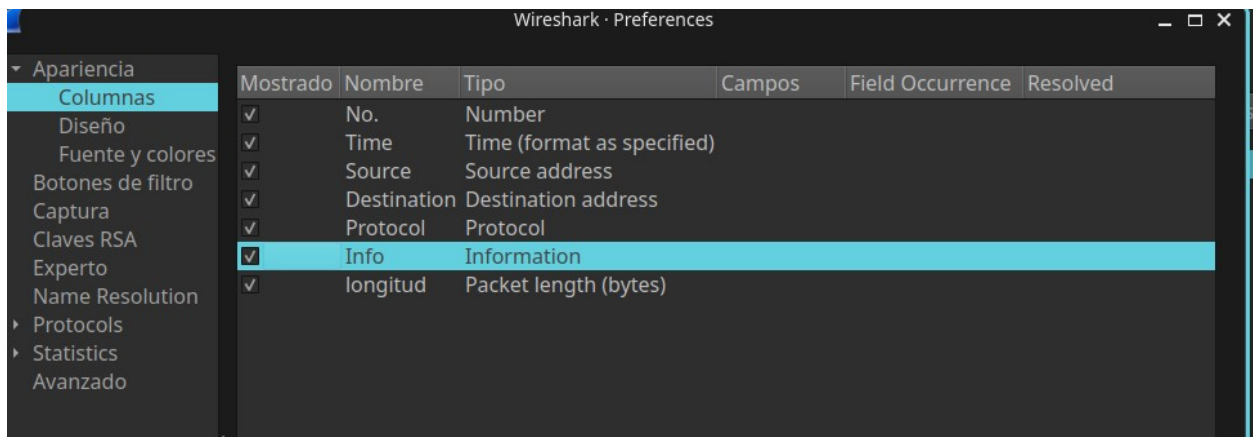
Formato de hora:

Segundos desde paquete mostrado anterior	Control+Alt+6
• Fecha y hora de día UTC (1970-01-01 01:02:03.123456)	Control+Alt+7
Año, día de año, y hora de día UTC (1970/001 01:02:03.123456)	

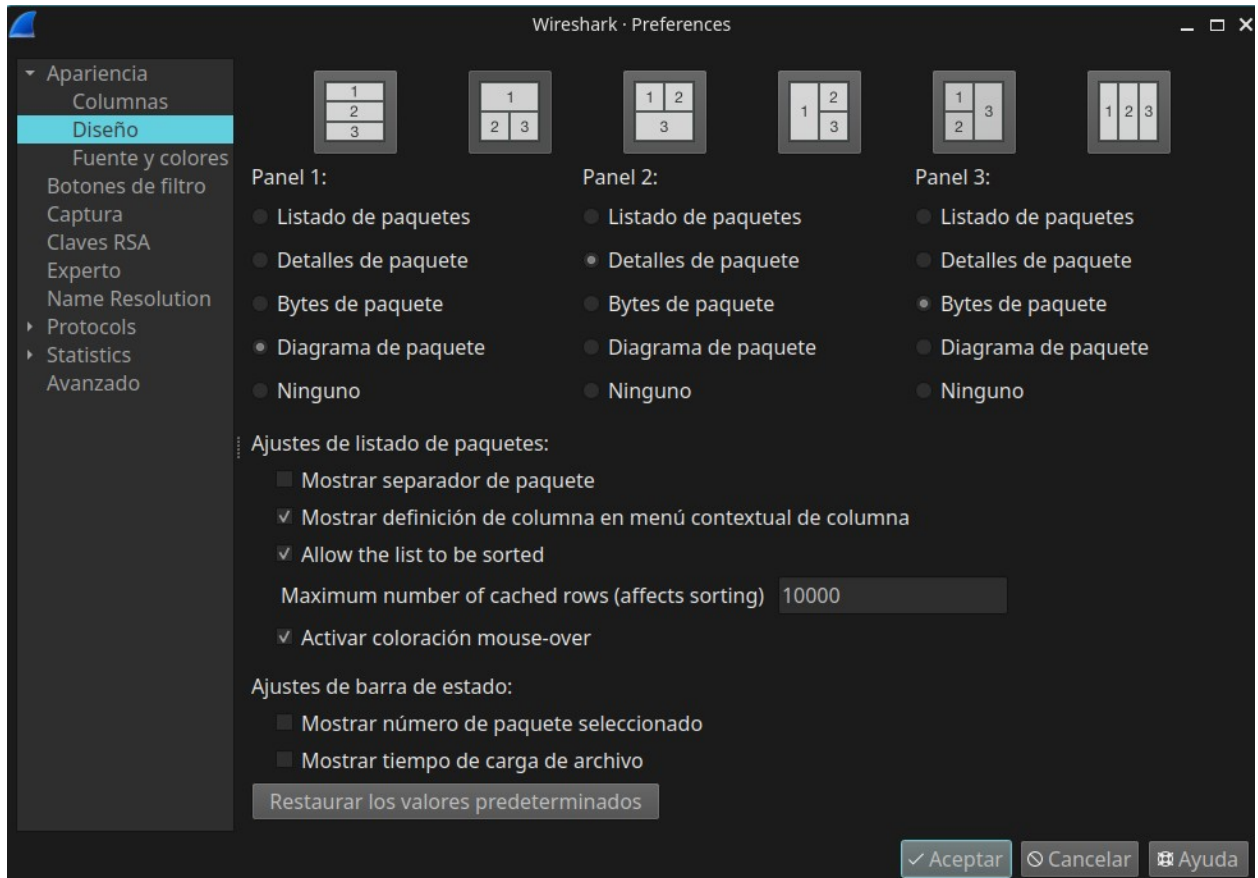
- Agregar columna de longitud del protocolo:



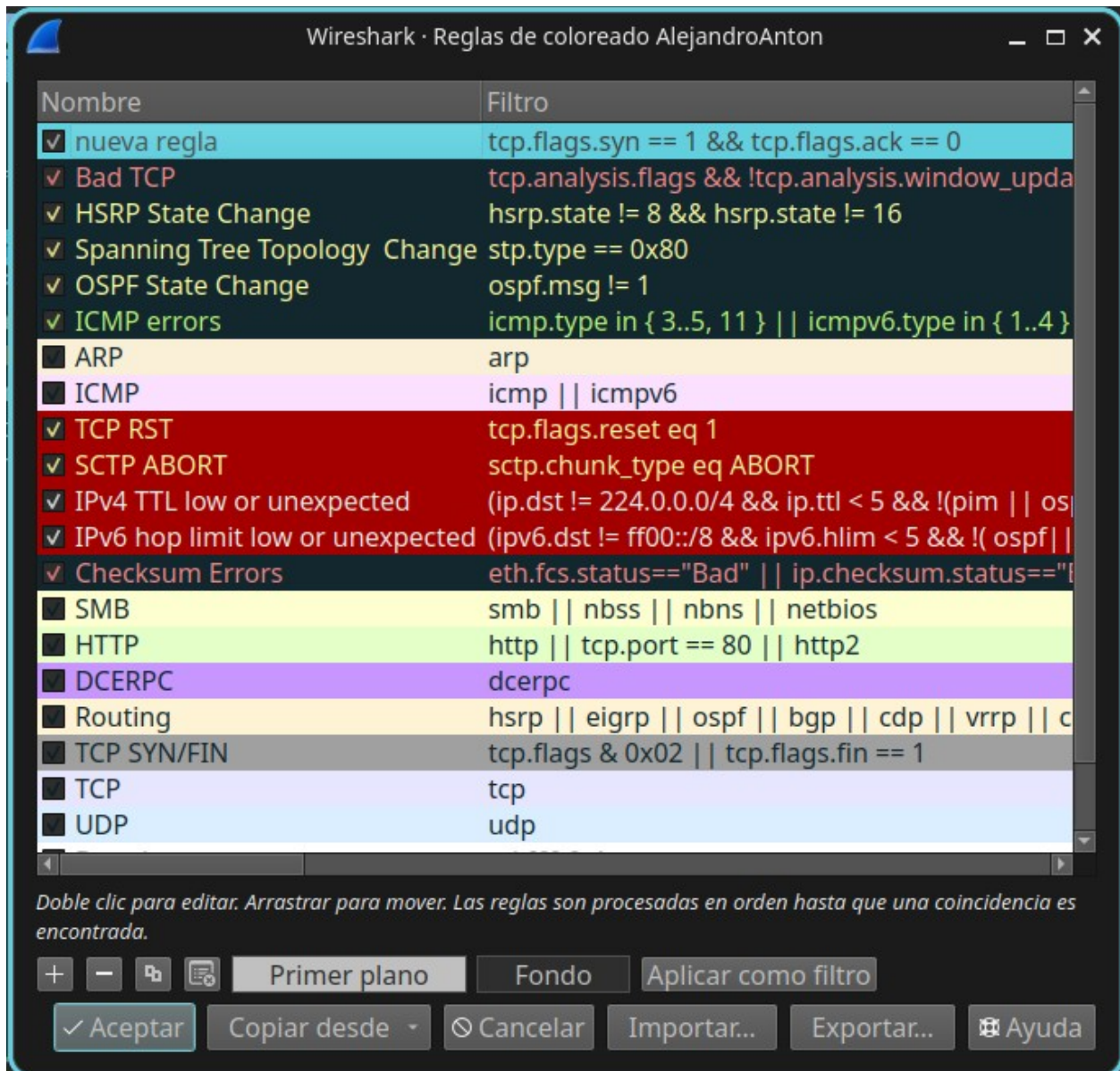
- Ocultar/eliminar columna Length por defecto



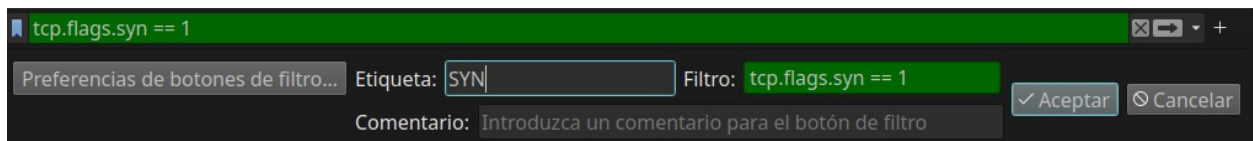
- Cambiar layout de paneles



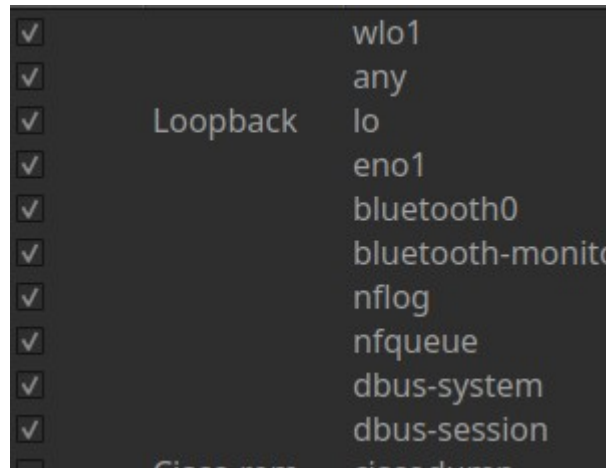
- Regla de color para SYN=1



- Botón de filtro rápido para SYN=1



- Ocultar interfaces virtuales



- ifconfig / ipconfig: ejecutar en terminal y documentar las interfaces activas, sus direcciones IP, máscara y estado (revisar 'man ifconfig' si hace falta explicar banderas como UP, BROADCAST, RUNNING, MTU).

```
janton17@pop-os:~$ ifconfig
eno1: flags=4099<UP,BROADCAST,MULTICAST> mtu 1500
    ether 6c:0b:5e:6d:75:c8 txqueuelen 1000 (Ethernet)
    RX packets 0 bytes 0 (0.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 0 bytes 0 (0.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Bucle local)
    RX packets 273 bytes 29793 (29.7 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 273 bytes 29793 (29.7 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

wlo1: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.1.7 netmask 255.255.255.0 broadcast 192.168.1.255
    inet6 fe80::7712:fb4b:a295:22f6 prefixlen 64 scopeid 0x20<link>
    inet6 2800:98:1622:4d14:8b19:8bd3:1563:ebe6 prefixlen 64 scopeid 0x0<global>
    inet6 2800:98:1622:4d14:c0d2:d178:b5eb:2c03 prefixlen 64 scopeid 0x0<global>
    ether 28:2e:89:1b:d6:18 txqueuelen 1000 (Ethernet)
    RX packets 13573 bytes 10526008 (10.5 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 9056 bytes 3918734 (3.9 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

janton17@pop-os:~$
```


3. Análisis de paquetes – Protocolo HTTP (3.6)

Captura realizada al acceder a <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>. Se identificaron 4 paquetes HTTP relevantes en la traza (intro-wireshark-trace1.pcap):

Frame	Origen → Destino	Método / Código	Longitud	Puerto
286	10.0.0.44 → 128.119.245.12	GET /wireshark-labs/INTRO-wireshark-file1.html	904 bytes	53962 → 80
288	128.119.245.12 → 10.0.0.44	200 OK	504 bytes	80 → 53962
290	10.0.0.44 → 128.119.245.12	GET /favicon.ico	861 bytes	53961 → 80
292	128.119.245.12 → 10.0.0.44	404 Not Found	551 bytes	80 → 53961

a. ¿Qué versión de HTTP está ejecutando el navegador?

El navegador (cliente, IP 10.0.0.44) envía la solicitud GET indicando HTTP/1.1, por lo que está ejecutando HTTP/1.1.

b. ¿Qué versión de HTTP está ejecutando el servidor?

El servidor (128.119.245.12, gaia.cs.umass.edu) responde con "HTTP/1.1 200 OK", por lo que también está ejecutando HTTP/1.1. Además, el encabezado Server indica que corre Apache/2.4.6 (CentOS) con OpenSSL/1.0.2k-fips, PHP/7.4.14, mod_perl/2.0.11 y Perl/v5.16.3.

c. ¿Qué lenguajes (si aplica) indica el navegador que acepta?

En el encabezado Accept-Language de la solicitud GET, el navegador indica: en-US, en;q=0.5. Es decir, prefiere inglés de Estados Unidos (en-US) y, como alternativa de menor prioridad (q=0.5), inglés genérico (en).

d. ¿Cuántos bytes de contenido fueron devueltos por el servidor?

El encabezado Content-Length de la respuesta 200 OK indica 81 bytes de contenido (el cuerpo HTML devuelto: "<html>\nCongratulations! You've downloaded the first Wireshark lab file!\n</html>"). Nota: el tamaño total del frame de respuesta es de 504 bytes, pero eso incluye encabezados Ethernet/IP/TCP/HTTP; el contenido (payload) propiamente dicho es de 81 bytes.

e. Rendimiento: ¿en qué elementos de la red convendría "escuchar" los paquetes? ¿Es conveniente instalar Wireshark en el servidor?

Si se sospecha un problema de rendimiento al descargar la página, conviene capturar tráfico en varios puntos de la ruta: en el host cliente (para medir el tiempo entre el envío de la solicitud y la llegada de la respuesta desde la perspectiva del usuario), en el(los) router(es) o enlaces intermedios de la red (para detectar retransmisiones, pérdida de paquetes o saturación de algún salto específico), y, de ser posible, cerca del servidor (para descartar que la lentitud se origine del lado del proveedor).

Instalar Wireshark directamente en el servidor de producción no suele ser recomendable: agrega carga de procesamiento y de E/S al propio servidor (justo el recurso que se busca diagnosticar), puede exponer tráfico sensible de otros usuarios que comparten ese servidor, y en muchos casos el administrador del servidor no es el mismo equipo que reporta el problema, por lo que no siempre se tiene el acceso o los permisos necesarios. Es más práctico capturar el tráfico en un punto de red cercano al servidor (por ejemplo, un puerto espejo/SPAN en el switch

del centro de datos) o directamente en el cliente/gateway, evitando así interferir con el rendimiento del servidor mientras se sigue obteniendo visibilidad del problema.

4. Discusión, experiencia y hallazgos

Trabajar con Wireshark permitió pasar de una idea abstracta de "la página se cargó" a observar exactamente qué paquetes se intercambiaron: dos solicitudes GET (una para el HTML principal y otra automática para favicon.ico), sus respectivas respuestas (200 OK y 404 Not Found), y todos los metadatos de encabezados HTTP que normalmente el navegador oculta al usuario final (cookies, user-agent, idiomas aceptados, tipo de servidor, etc.). Resultó interesante confirmar que incluso una página tan simple como la del laboratorio genera más de una transacción HTTP, y que analizar los encabezados de cada respuesta da pistas útiles de diagnóstico (por ejemplo, el Content-Length permite verificar si el navegador recibió el contenido completo).

La combinación de la primera parte del laboratorio (esquemas de comunicación manuales) con esta segunda parte (análisis de tráfico real) ayudó a conectar la teoría de codificación y conmutación con lo que realmente ocurre a nivel de paquetes: así como en la actividad de conmutación había que indicar el destino del mensaje antes de enviarlo, cada paquete IP/TCP capturado incluye direcciones de origen y destino, y el protocolo HTTP en sí sigue un esquema de petición/respuesta bien definido.

5. Conclusiones

Wireshark demostró ser una herramienta poderosa para observar el comportamiento real de un protocolo de aplicación (HTTP) sobre TCP/IP, revelando información que normalmente permanece oculta al usuario: versiones de protocolo, servidores web utilizados, cookies, idiomas aceptados y tamaños exactos de contenido transferido.

Se concluye también que la ubicación desde donde se captura el tráfico es determinante para el diagnóstico de problemas de rendimiento, y que instalar un analizador de paquetes directamente en un servidor de producción no siempre es la mejor práctica por razones de carga adicional, privacidad y control de acceso.

6. Referencias

Kurose, J. F., & Ross, K. W. Wireshark Lab: Getting Started. Recuperado de <http://gaia.cs.umass.edu/wireshark-labs/INTRO-wireshark-file1.html>

Universidad del Valle de Guatemala, Departamento de Ciencias de la Computación. (2026). Laboratorio #1 – Esquemas de comunicación e introducción a Wireshark. CC3067 - Redes.

Documentación oficial de Wireshark: <https://www.wireshark.org/docs/>